Slide 1

Toegepaste Informatica

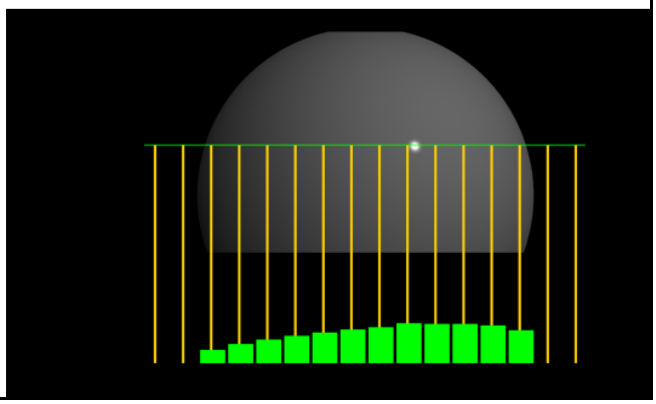
2024

Week 8: Advanced pipeline

Slide 2

Last class

- Aliasing



Slide 3

Course schedule

Highly likely to change ☹

		Wed
Week 1		Introduction
Week 2		3D reasoning
Week 3		Geometry
Week 4		Pipeline & Materials
Week 5		Proceduralism
Week 6		Textures & Normal maps
Week 7		Aliasing / Presentations
Week 8	Lab	Advanced pipeline 1 Lab
Week 9	Advanced pipeline 2 Lab	Post Processing
Week 10	?	Project
Week 11	?	Revision

Slide 4

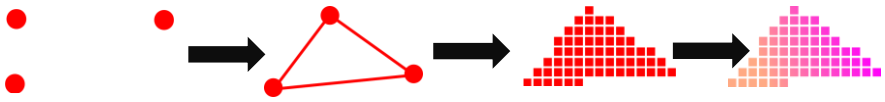
This class

- → Pipeline recap ←
- Back in the day
- Light & Shadow
- Camera
- Ray Tracing vs Rasterization
- Forward rendering

Slide 5

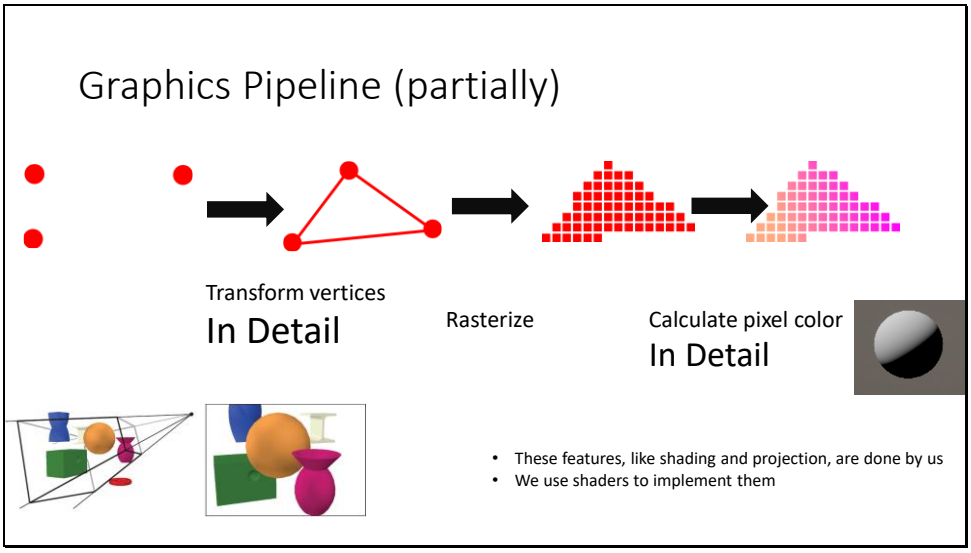
GPU Pipeline

- Remember, the GPU pipeline was a series of steps (and hardware) developed for rendering
- It's very low level. It allows us to do rendering, but doesn't do many features. That's for engines (like Unity)
- For example, it doesn't do lights, shadows, camera's. It does matrices, vectors, textures



Vertex Shader Rasterize Calculate pixel color

Slide 6



It's partial, because there's a lot of stuff happening when we render an image that doesn't fit in our vertex->rasterization->pixel pipeline. An example would be color correction

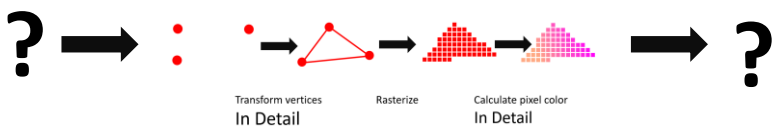
In detail, I mean that we use the shaders for actual graphics tasks, like transform our mesh into view space, or shading. This is also part of the pipeline, but not of the graphics api

Slide 7

Graphics Pipeline

- There's a ton more things that we can't just fit in the vertex of pixel shader of an object
- How does fog work?
- How does HDR work?

- There's more to the pipeline than rendering a single object

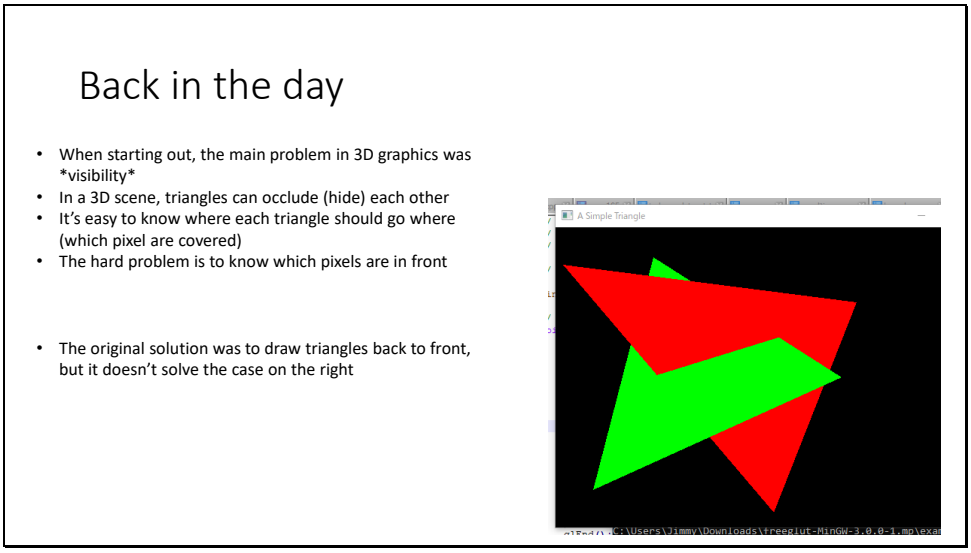


Slide 8

This class

- Pipeline recap
- → Back in the day ←
- Light & Shadow
- Camera
- Ray Tracing vs Rasterization
- Forward rendering

Slide 9



This is going on a tangent, but becomes relevant later

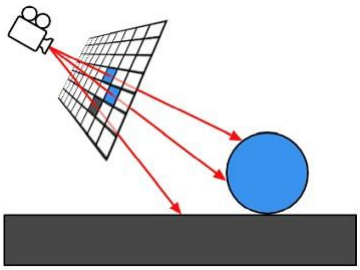
In the early days of rendering, they faced a difficult problem. They could rasterize triangles by writing to a grid, just like you would paint them on a wall: first the one in back, then the one covering it.

What they could not due, was solve the problem where one wasn't always completely behind the other.

Slide 10

Raytracing

- Store all the triangles in a data structure
- For each pixel, create a ray
- “shoot” the ray, finding the intersections with triangles
- Take the **closest** intersection

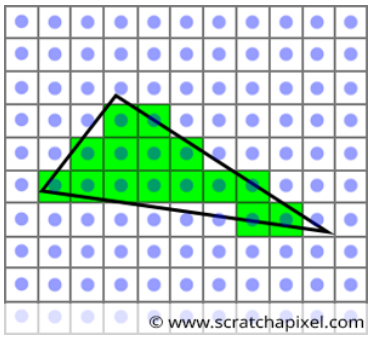


Slide 11

Rasterization

- For each triangle
- Project triangle onto grid
- Calculate which pixels are covered (center)

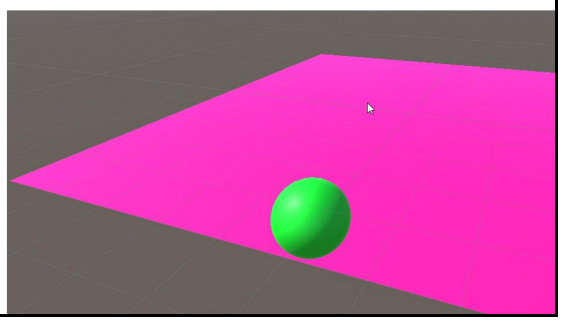
- That doesn't solve the visibility problem



Slide 12

Primary visibility : rasterization

- Objects are drawn on top of eachother
- Eg closest to camera
- Last object “overwrites” pixels
- Like a painting
- In this case, the plane is penetrating the ball



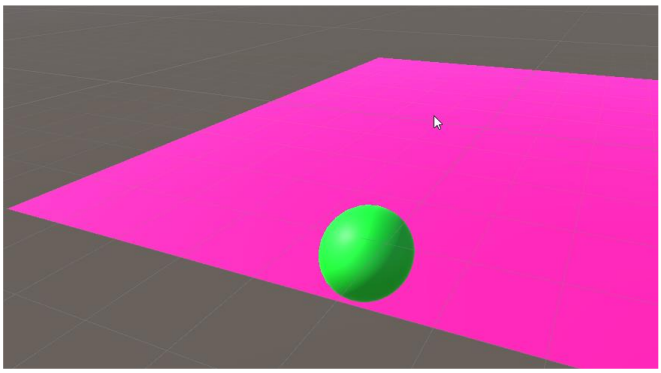
We render a ball that sinks into the plane

We render the plane first, and then the ball

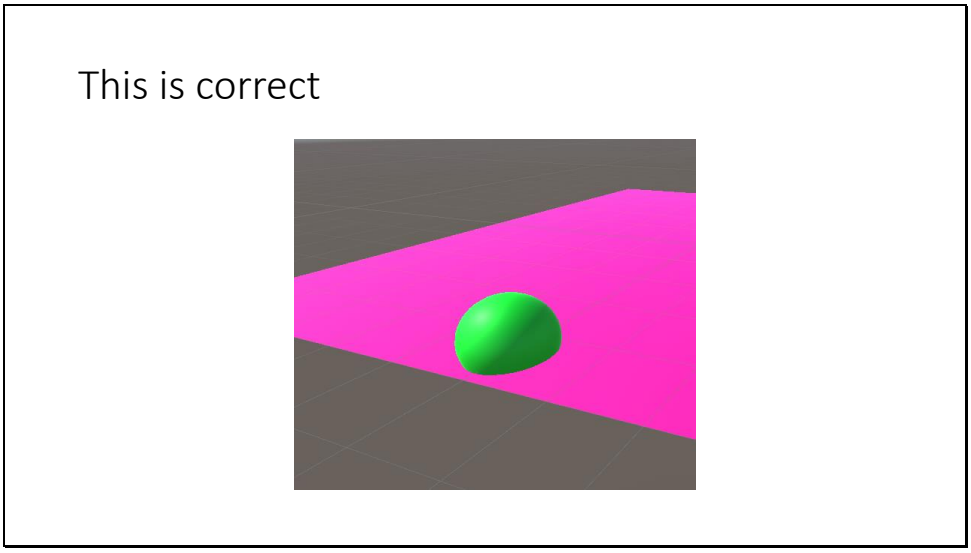
But now the ball looks like it’s standing on the plane

Slide 13

This is not correct



Slide 14

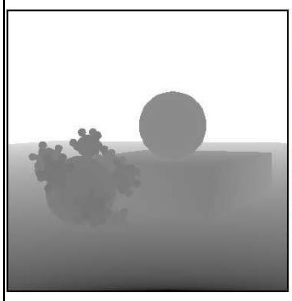


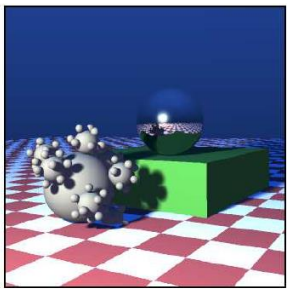
Part of the plane is behind the ball

Part of the plane is in front of the ball

Slide 15

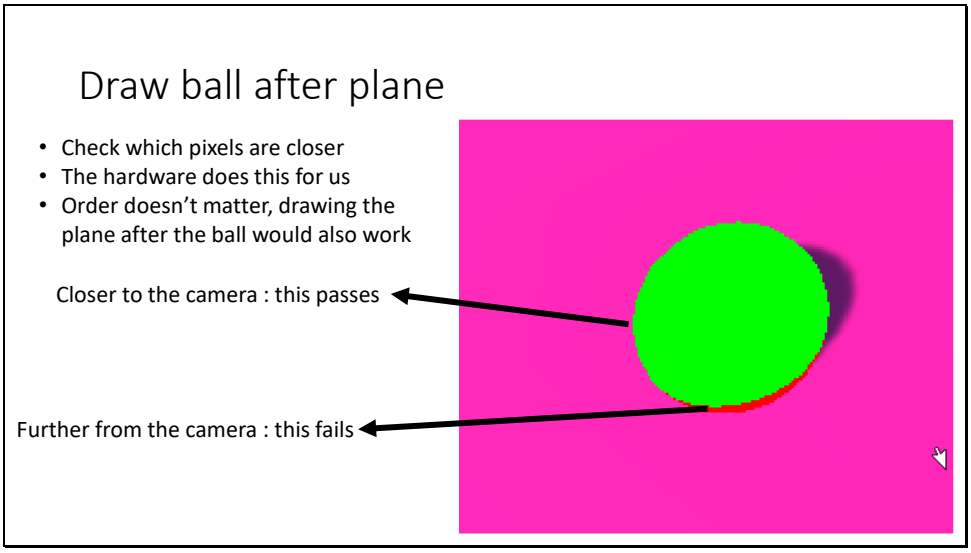
Depth buffer





- Separate texture
- Stores the depth (distance to camera) of pixel writing color
- If a new pixel is further than the depth value : skip
- If a new pixel is closer than the depth value : write depth, write color

Slide 16



We're now drawing the ball

The green pixels are closer to the camera than the plane in the same pixel position, so the ball color is set in that pixel

The red pixels have the ball further from the camera than the plane. We're going to skip it and just keep the pixel from the plane

Slide 17

This class

- Pipeline recap
- Back in the day
- → Light & Shadow ←
- Camera
- Ray Tracing vs Rasterization
- Forward rendering

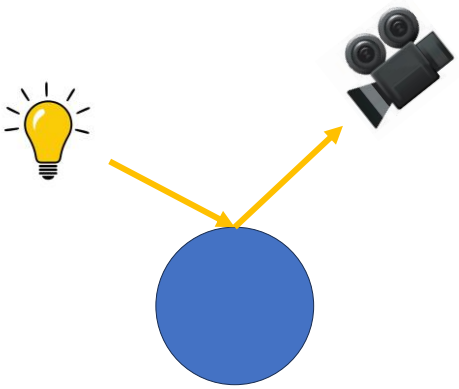
Slide 18

Quick detour, it’ll make sense after?

Slide 19

Light transport

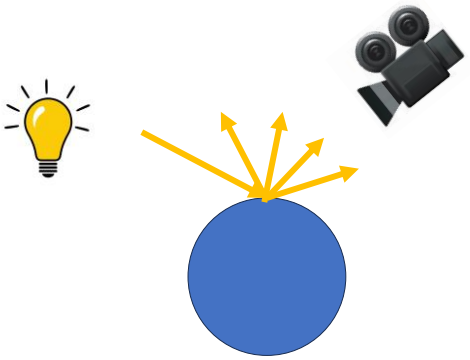
- Light hits an object
- That light **bounces**
- It enters our eye



Slide 20

Light transport

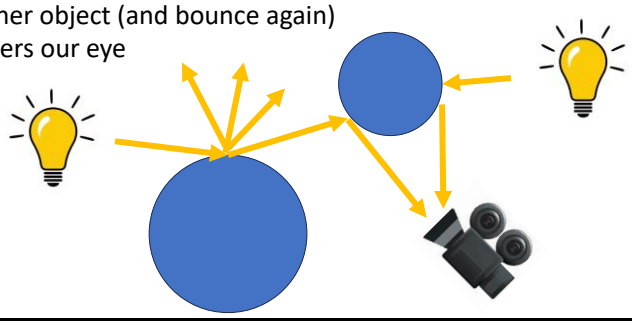
- Light hits an object
- That light **bounces**
- In many directions
- Some materials more than others
- Some of it enters our eye



Slide 21

Light transport

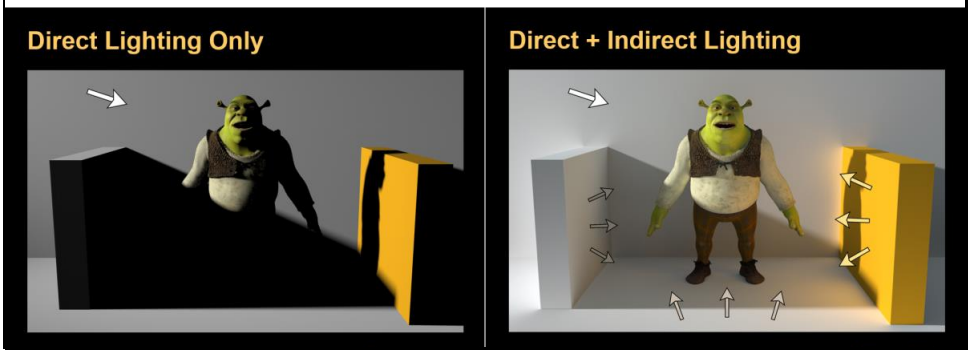
- Light hits an object
- That light **bounces**
- In many directions
- Most hit another object (and bounce again)
- Some of it enters our eye



Slide 22

This is called global illumination

- Most lighting comes directly from a light source (lamp, sun, sky)
- It's reflected on the surface
- Some of it hits the camera, some of it reflects off of other surfaces
- Some of that reflected light will get reflected again towards the camera
- We're not going to look at it, but it's good knowledge for the next section



Next problem: secondary visibility

- The visible point (pixel on screen) needs to be shaded (BRDF)
- For that, we need to know where the lights are
- We also need to know where the shadows are?

Slide 24

What are shadows anyway?

Slide 25

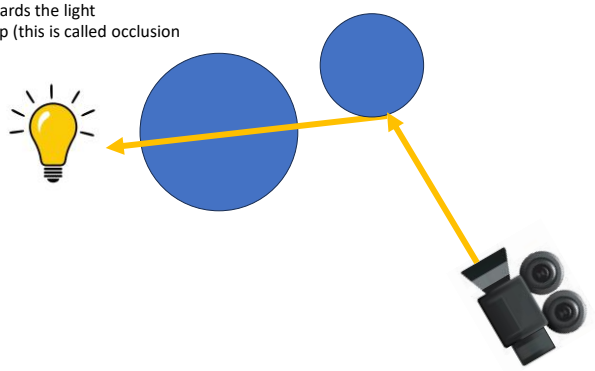
What are shadows anyway?

- Shadows are places on a surface a light *doesn't* reach
- They're a hard problem
- Unity does it for us (same as with lights and camera's)
- But let's look at how they're made, it's interesting

Slide 26

Raytracing : easy

- For each light, shoot a ray towards the light
- If there is geometry closes, skip (this is called occlusion

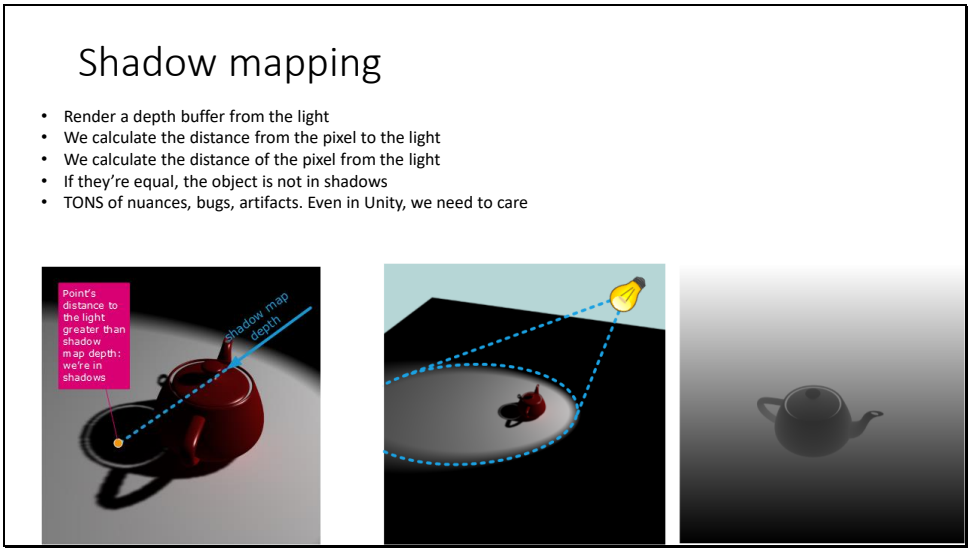


Slide 27

Rasterization : hard

- Same idea
- But we can't shoot rays
- How can we tell if light or geometry is closer?
- We solved a similar problem this class

Slide 28

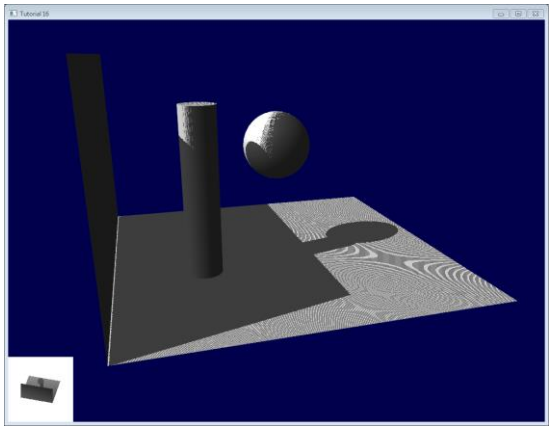


Right : depth buffer from the position of the light. This tells us how far thigs are from the light
Middle: we're shading a pixel, the one with the orange color. We know it's worldspace position, and, using a transform, know the position from the pov of the light. So we know how far it is from the light. That gives us the information if something is even closer (here, the teapot), and is occluding (casting a shadow) the floor.

Slide 29

Shadow mapping : problems

- What problems do you see?
- I see at least 3



“Shadow acne” where the pixels flip on and off on the floor
“Peter panning”, where the shadow of the wall doesn’t properly hit the wall

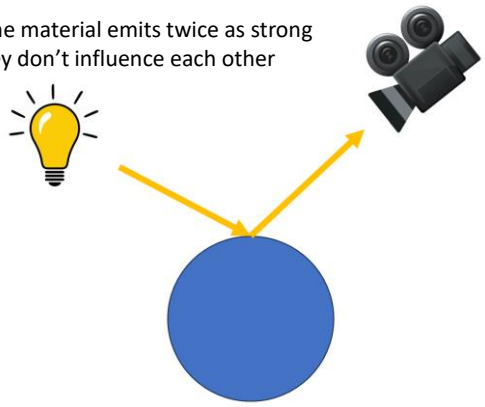
Both are caused by floating point precision issues

And also, the edge of the big shadow plane is very jagged, which is due to low-quality texture filtering

Slide 30

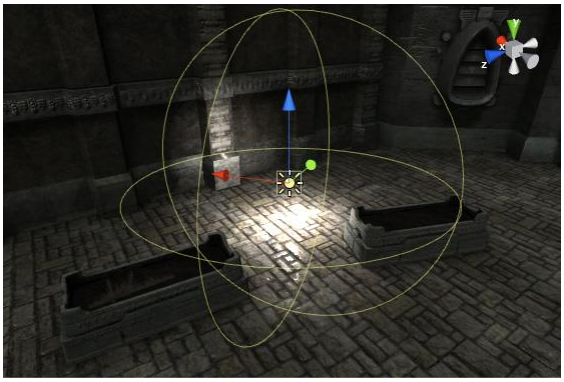
Light is and additive

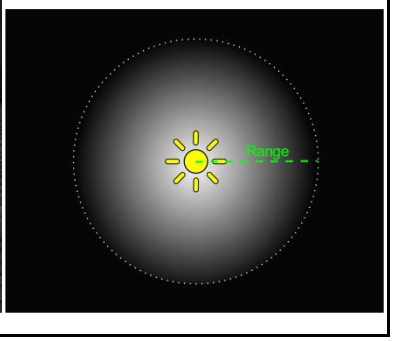
- If we make the light twice as strong, the material emits twice as strong
- We can treat two lights separately, they don't influence each other



Slide 31

Point lights

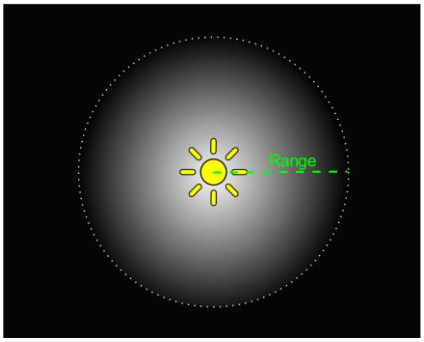




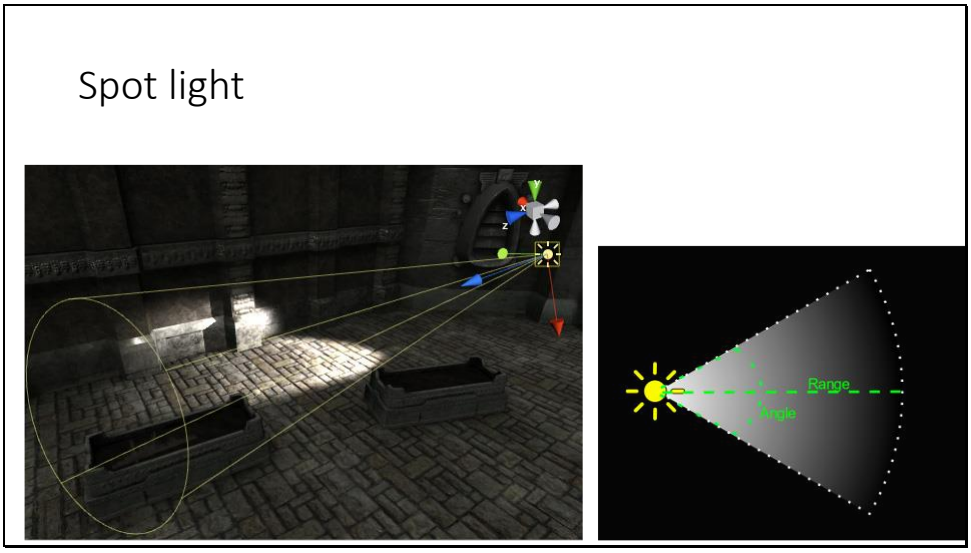
Slide 32

Attenuation

- Describes how light get less powerful with distance (with a cutoff)



Slide 33

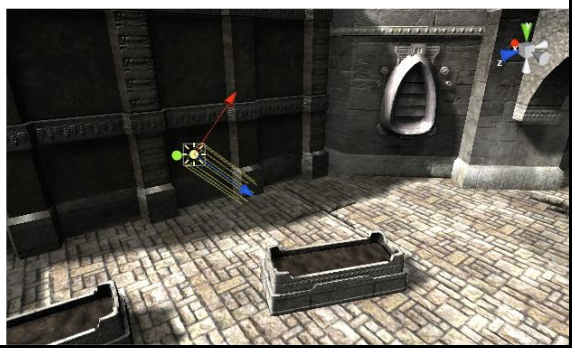


Same as point light, but with two angles

Slide 34

Directional light

- Like sunlight
- Does not fall off with distance
- Why?

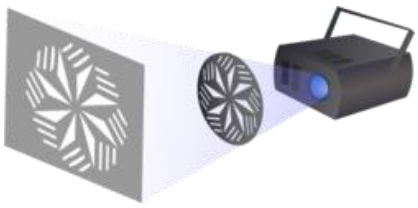


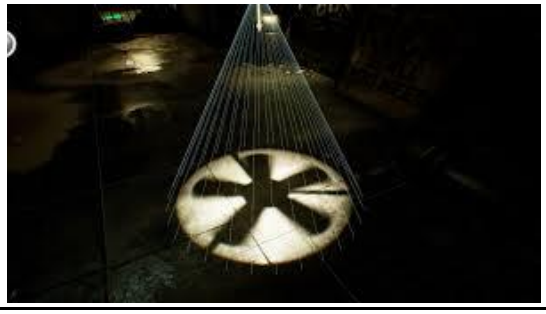
The light doesn’t spread out, like it does for point and spot lights.

Slide 35

Light cookies

- Like a painted plastic film on a light
- Cheap way to create detail





Slide 36

This class

- Pipeline recap
- Back in the day
- Light & Shadow
- → Camera ←
- Ray Tracing vs Rasterization
- Forward rendering

Slide 37

Perspective camera

- Objects get smaller the further they are (depth cue)
- All real cameras are perspective camera's

Slide 38

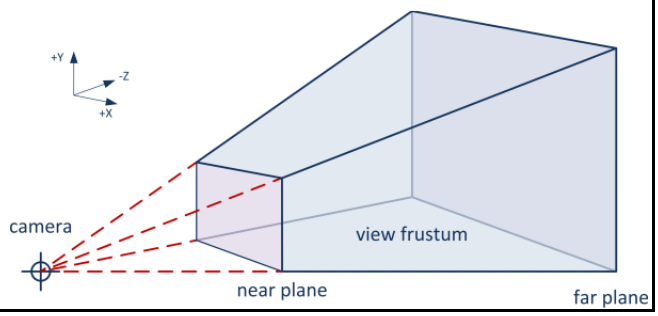

34
AMMO
45%
HEALTH
2 3 4
5 6 7
ARMS
144%
ARMOR
BULL 30 / 400
SHEL 44 / 100
ROKT 34 / 100
CELL 20 / 600

Slide 39

Slide 40

Perspective camera : frustum

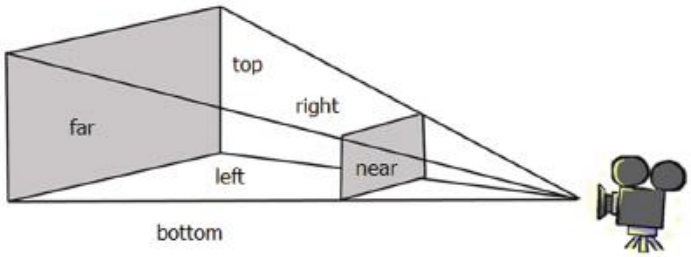
- The camera has a frustum (capped pyramid)
- Anything visible has to be in the frustum



Slide 41

Perspective camera : near / far

- The closest cutoff is called the near plane
- The furthest cutoff is called the far plane
- We need the cutoff due to the depth buffer



Slide 42

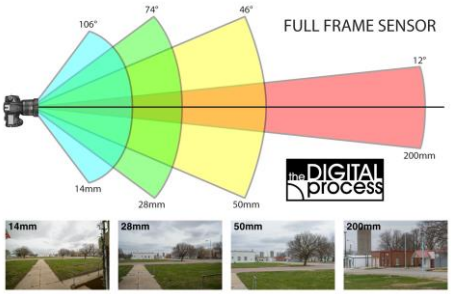
Perspective camera : field of view

- This might already be familiar to you?

Slide 43

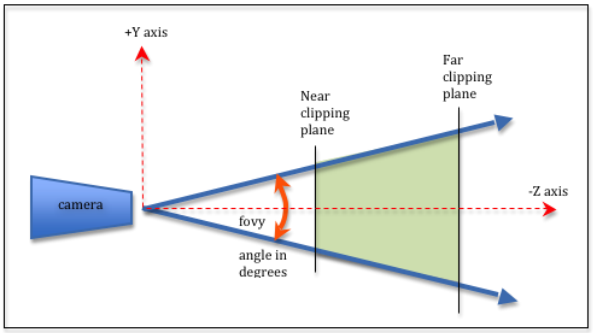
Analogous to different lenses

Larger field of view : smaller objects



Slide 44

Field of view in frustum



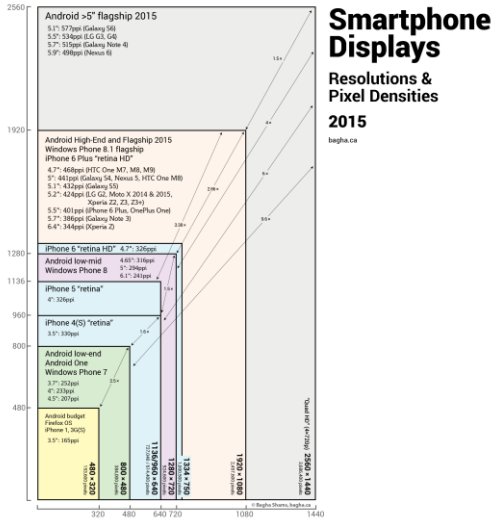
Field of view is determined by aspect ratio

- Aspect ratio : resolution.x / resolution.y
- Eg 1920 * 1080
- Common (nowadays) : 16/9

- For FOV, we usually choose the vertical one, eg 90 degrees
  Multiply by aspect ratio for horizontal one : (16 * 90) / 9
  FOV = (160, 90)

Slide 46

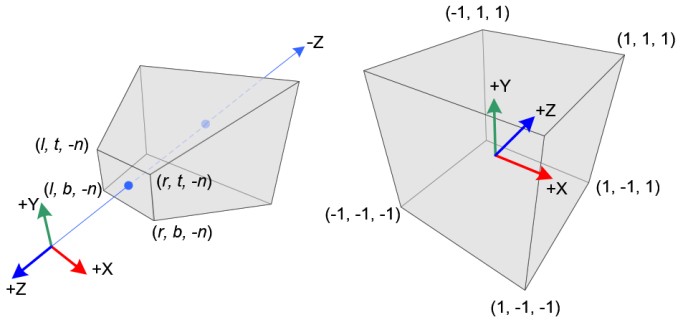
Phones are a nightmare



Slide 47

Perspective matrix

- Takes care of perspective transform
- Comes after the view matrix
- Common : MVP (ModelViewProjection)



Slide 48

Questions?

Slide 49

Camera's

- Perspective
- →Orthographic←

Slide 50

Orthographic

- Objects do not get smaller as they get further
- Does anyone know an example?

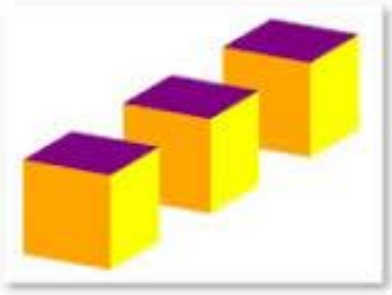
Slide 51

Slide 52

Slide 53

Comparisio n1

Orthographic Projection

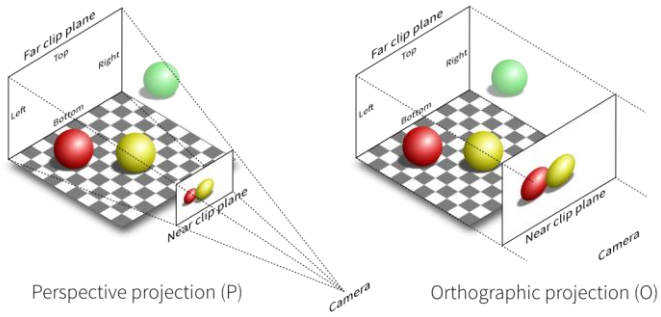


Perspective Projection

Slide 54

Comparison 2

- Very similar to perspective
- No fov
- Near (and far) have width and hight



Perspective projection (P)

Orthographic projection (O)

Slide 55

Questions?

Slide 56

Questions?

Slide 57

This class

- Pipeline recap
- Back in the day
- Light & Shadow
- Camera
- → Ray Tracing vs Rasterization ←
- Forward rendering

Slide 58

Pros and cons

Raytracing

- Good quality (multiple rays per pixels)
- Easy implementation (at the start)
- Manages smoke, mirrors
- Bad performance

Rasterization

- Ok quality (one fragment per pixels)
- Hard implementation (at the start
- Difficult smoke, mirrors
- Good performance

* Lots of simplifications and lies on this slide

Pros and cons

Raytracing

- Good quality (multiple rays per pixels)
- Easy implementation (at the start)
- Manages smoke, mirrors

- Bad performance

Rasterization

- Ok quality (one fragment per pixels)

- Hard implementation (at the start
- Difficult smoke, mirrors

- Good performance

* Lots of simplifications and lies on this slide

Slide 60

Pros and cons

Raytracing

- Good quality (multiple rays per pixels)
- Easy implementation (at the start)
- Manages smoke, mirrors

- Bad performance

Rasterization

- Ok quality (one fragment per pixels)

- Hard implementation (at the start
- Difficult smoke, mirrors

- Good performance

* Lots of simplifications and lies on this slide

Slide 61

Pros and cons

Raytracing

- Good quality (multiple rays per pixels)
- Easy implementation (at the start)
- Manages smoke, mirrors

- Bad performance

Rasterization

- Ok quality (one fragment per pixels)

- Hard implementation (at the start
- Difficult smoke, mirrors

- Good performance

* Lots of simplifications and lies on this slide

Performance is king in realtime rendering. If we can render a scene faster, we can add more detail, making it slower, so we make it faster….

Slide 62

Forward rendering

- So, we go with rasterization
- We get our vertex shader / pixel shader
- How do we implement lights, camera's and all that?

Slide 63

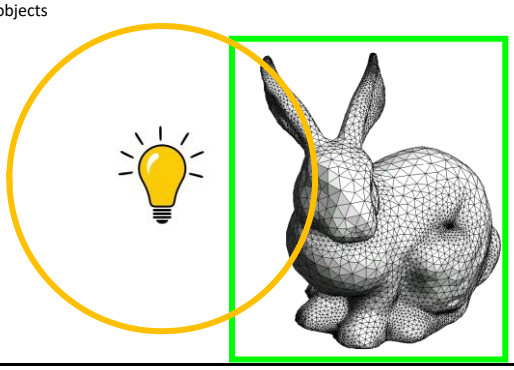
Gather objects : frustum culling

- Instead of rendering all objects, render only the ones in the camera frustum
- Not necessary, but it's cheap and gives us a lot of performance
- Unity does this for us

Slide 64

For each object, find lights

- Done using bounds tests
- Often with an acceleration structure (100's of lights, 1000's of objects)
- Find lights overlapping objects



Slide 65

For each object, render

Vertex shader

(Apply animation)
Apply mvp matrix

Fragment shader

Light lights; (from cpu)

```
Float4 outputColor = 0;
For each light:
  if (inShadow(shadowMap))
     continue
  outputColor += BRDF(N, light) *
Return outputColor
```

Slide 66

BRDF Function

- Very free
- Can use color maps, or not
- Can use normal maps, or not,
- Gooch, toon, blinn-phong…

BRDF (one light)

```
Float3 outputColor = albedo; // baseColor
outputColor *= light.Intensity / distance(light.pos,
vertex.pos); // attenuate
outputColor *= light.color;
outputColor *= sampleShadow(light, pos);
outputColor *= dot(N, normalize(pos – light.pos))
Return outputColor;
```

Slide 67

Possible Exam questions

- What is primary visibility?
- What is the depth buffer? What problem does it solve?
- What is a perspective / orthographic camera? Give an example usecase for each

Slide 68

Next week

- Deferred rendering
- Used in virtually all games after 2007
- Will look at a renderdoc capture of unity / AAA game